

Policy Engine Summary

Overview

We have successfully implemented the **Decide** stage of the RecoverAI pipeline by deploying the **Policy Engine**. This module acts as the "brain" of our operation, taking the diagnosed failures and matching them against our business logic to determine the most effective and compliant recovery action.

Achievements

1. Stopping Rules Configuration (`config/stopping_rules.json`)

- Formalized the intervention mapping for our 5 root-cause buckets:
 - `insufficient_funds` → `wait_and_retry`
 - `bank_decline` → `send_payment_link_with_alt_method`
 - `abandoned_checkout` → `send_discounted_payment_link`
 - `mandate_failure` → `notify_and_recreate_mandate`
 - `overdue_invoice` → `promise_to_pay_tracker`
- Added global stopping rules, including maximum retry attempts, minimum cool-off windows (in hours), and timezone-aware Do-Not-Disturb (DND) window checks.

2. Policy Evaluation Logic (`src/policy_engine.js`)

- Built the engine to query `pending_policy_evaluation` cases from the database.
- For every case, the engine:
 1. Maps the root cause to the defined configuration.
 2. Validates against the `max_attempts` ceiling.
 3. Ensures the `cool_off_hours` duration has passed since the last intervention.
 4. Blocks actions during DND hours if the bucket rule strictly demands compliance (e.g., sending an SMS for an abandoned checkout).
- Successfully processed the 75 simulated recovery cases, determining a bounded action (`action_determined`) or a `wait` command for each, and appending the reasoning to the per-case JSON Audit Trail.

Next Steps

With our cases cleanly mapped to explicit recovery actions, we are ready to move into the **Execute** stage. The upcoming Execution Layer will translate these decisions into real Razorpay API calls via our MCP server and simulate customer communications.